CSD201 PE INSTRUCTIONS

Read all the instructions below carefully before you start coding.

Follow the steps below to complete PE:

1. Create a folder to store the given projects, e.g. CSD_given (1). Download the provided materials into (1).
2. Fulfill all the requirements of the test.
3. **Before submitting:**
   - Clean and Build Project (Shift+F11),
   - Then rename the **dist** folder to **run**. (If the run folder already exists, delete it before renaming).
4. **Submitting your work:**
   - To submit project Q1, first select the Question No: 1
   - Browse and pick the project folder (e.g. Q1, or Q1A, or Q2, ...)
   - Then click the **Submit** button.
5. **Avoid using accented Vietnamese** when writing comments in your programs.
6. Your job is to **complete only the method bodies**. **Do not** alter method signatures or add new import statements.
7. The software tools required are **NetBeans IDE 13** and **Java JDK 8 or above**. **If any of the above requirements is not followed, the exam will get ZERO.**

Notes:

- The input and expected output below are provided **only to test your code**.
- The input and expected output in the **real test cases** (used for grading) are **different** from those in the examples below.
- **Do not hardcode** the given expected results.

Troubleshooting: If the given project (e.g. Q1) fails to run, run "Clean and Build Project" (Shift+F11). If the error persists, try renaming or copying the project to another name, e.g. from Q1 to Q1X or Q1Y.

Questions: (10 marks)

Scenario: You will complete a Java program that simulates a **Task Management System**. This system uses a **Binary Search Tree** and a **Queue** to manage a set of given tasks.

Class Structure:

- **Task**: Stores the data of a single task: **taskId** (String), **description** (String) and **priority** (int)
- **TreeNode**: Holds a Task object, a left pointer and a right pointer for the Binary Search Tree structure.
- **QueueNode**: Holds a Task object and a next pointer for the Queue structure.
- **TaskBST**: A **Binary Search Tree** structure that stores the given tasks in the normal BST order based on **priority**.
- **Completion Queue**: A **Queue** structure (implemented as a linked list) used to manage the given tasks in FIFO (First-In, First-Out) order.

You must read the provided code segments carefully to fully understand the relationships between the classes and the functions inside each class.

1. f1() – 2.5 marks: Implement Basic Queue Operations

- **File:** CompletionQueue.java
- **Methods to Complete:** enqueue(Task task), isEmpty()
- **Task:** Implement two basic operations for a queue. isEmpty() must return true when the queue is empty, and enqueue() must add a new Task to the back (rear) of the queue.

Expected example output:

```
>>> Testing f1: CompletionQueue isEmpty() and enqueue()
Is queue empty initially? Yes
Enqueueing two tasks...
Is queue empty now? No

Final Queue State:
--- Completion Queue ---
(C01, Completed_A, P:99)
(C02, Completed_B, P:100)
```

2. f2() – 2.5 marks: Insert a Task into the Binary Search Tree

— Tài liệu được thành viên đăng tải tại ***FUOVERFLOW*** và được biên soạn lại để hỗ trợ ôn tập —

- **File:** TaskBST.java
- **Method to Complete:** insert(Task task)
- **Task:** Write the logic to insert a new Task object into the Binary Search Tree. The tree must be ordered based on the priority of the task. If a task with the same priority

already exists, the new task can be ignored. The output is printed in in-order traversal.

Expected example output:

```
>>> Testing f2: Inserting tasks into BST...

--- Final State ---
--- Task BST ---
(T05, Write_Documentation, P:4)
(T02, Update_UI, P:5)
(T04, Test_New_Feature, P:7)
(T03, Fix_Login_Bug, P:8)
(T06, Refactor_Database, P:9)
(T01, Deploy_Server, P:10)
```

3. f3() – 2.5 marks: Pre-Order Traversal of the BST

- **File:** TaskBST.java
- **Method to Complete:** preOrderTraversal()
- **Task:** Implement a method that traverses the BST in pre-order. This method should take no parameters and must return a java.util.List containing all the tasks from the tree. A private recursive helper method is a good approach for this task.

Expected example output:

```
>>> Testing f3: Traversing the BST pre-order...

--- Final State ---
--- Task BST ---
(T03, Fix_Login_Bug, P:8)
(T05, Write_Documentation, P:4)
(T02, Update_UI, P:5)
(T04, Test_New_Feature, P:7)
(T01, Deploy_Server, P:10)
(T06, Refactor_Database, P:9)
```

4. f4() – 2.5 marks: Remove a Task from the BST

- **File:** TaskBST.java
- **Method to Complete:** remove(int priority)
- **Task:** Write the logic to remove a task from the BST based on its priority. You must correctly handle all three deletion cases: a node with no children, one child, or two children. You may use as many helper functions as you want.

Expected example output:

```
--- Initial State ---
--- Task BST ---
(T05, Write_Documentation, P:4)
(T02, Update_UI, P:5)
(T04, Test_New_Feature, P:7)
(T03, Fix_Login_Bug, P:8)
(T06, Refactor_Database, P:9)
(T01, Deploy_Server, P:10)

>>> Testing f4: Removing task with priority 4...

--- Final State ---
--- Task BST ---
(T02, Update_UI, P:5)
(T04, Test_New_Feature, P:7)
(T03, Fix_Login_Bug, P:8)
(T06, Refactor_Database, P:9)
(T01, Deploy_Server, P:10)
```